AI LAB 2

Experiment 5: Solving the 8-Queens Problem Using Backtracking

AIM: To find the collision-free placement of components on a grid.

QUESTION:

An automated placement system must position eight components on an 8 × 8 grid such that no two components interfere with one another. Model this as the 8-Queens problem, where each queen represents a component and can attack another queen along the same row, column, or diagonal.

Develop a Python program using the backtracking algorithm to find a valid arrangement of eight queens on the chessboard.

The program must:

1. **Represent the chessboard as an 8 × 8 grid and track the positions of the queens.**
2. **Place one queen at a time while ensuring that no two queens share the same row, column, or diagonal.**
3. **Use recursive backtracking to undo an invalid placement and explore alternative positions.**
4. **Display the final chessboard arrangement and the positions of all eight queens.**
5. **Report the number of placement attempts and explain how constraint checking and backtracking help find a valid solution.**

Code:

```
N = 8

# Create an 8 x 8 chessboard
board = [[0 for _ in range(N)] for _ in range(N)]

# Store the column position of each queen
positions = []
```

```
# Count placement attempts
attempts = 0


def is_safe(row, col):
    # Check the same column in previous rows
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check upper-left diagonal
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check upper-right diagonal
    i, j = row - 1, col + 1
    while i >= 0 and j < N:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True


def solve(row):
    global attempts

    # All eight queens have been placed
    if row == N:
        return True

    # Try placing a queen in each column
    for col in range(N):
        attempts += 1
```

```
        if is_safe(row, col):
            # Place the queen
            board[row][col] = 1
            positions.append((row + 1, col + 1))

            # Solve the next row recursively
            if solve(row + 1):
                return True

            # Backtrack: remove the queen
            board[row][col] = 0
            positions.pop()

    return False


# Execute the algorithm
if solve(0):
    print("Valid 8-Queens Solution:\n")

    # Display the chessboard
    for row in board:
        print(" ".join("Q" if cell == 1 else "." for cell in row))

    print("\nQueen Positions (Row, Column):")
    for position in positions:
        print(position)

    print("\nTotal placement attempts:", attempts)
else:
    print("No solution exists.")
```

Output:

```
Valid 8-Queens Solution:

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .

Queen Positions (Row, Column):
(1, 1)
(2, 5)
(3, 8)
(4, 6)
(5, 3)
(6, 7)
(7, 2)
(8, 4)


Total placement attempts: 876
```

Experiment 6: Solving the Map Colouring Problem Using CSP

AIM: To make frequency allocation for neighbouring cellular network regions.

QUESTION:

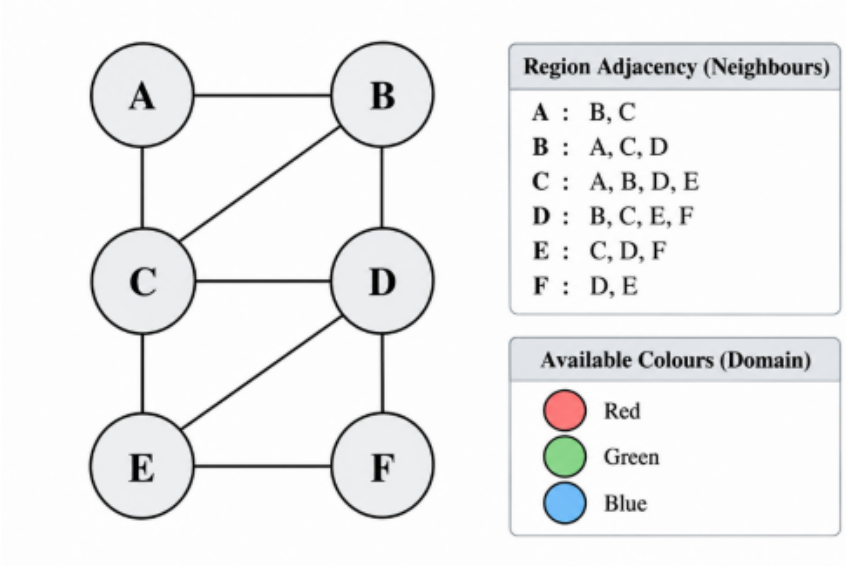
A telecommunications network is divided into interconnected regions. To reduce interference, neighbouring regions must be assigned different frequency bands. Model the network as a Constraint Satisfaction Problem (CSP), where each region is a variable, the available frequency bands form its domain, and neighbouring regions must receive different assignments.

Develop a Python program to solve the map-colouring problem using CSP techniques.

The program must:

1. **Represent each region as a variable and define its available colours as a domain.**
2. **Specify adjacency constraints requiring neighbouring regions to have different colours.**
3. **Use backtracking search to find a valid colour assignment.**
4. **Display the final map-colouring assignment and verify that all adjacency constraints are satisfied.**

Report the number of assignments attempted and explain how CSP variables, domains, and constraints guide the search.



Code:

```
# Experiment 6: Map Colouring using CSP and Backtracking

# Available colours (domains)
colours = ["Red", "Green", "Blue"]

# Variables: regions A, B, C, D, E, F
regions = ["A", "B", "C", "D", "E", "F"]

# Adjacency constraints from the given diagram
neighbours = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D", "E"],
    "D": ["B", "C", "E", "F"],
    "E": ["C", "D", "F"],
    "F": ["D", "E"]
}

# Store the assigned colours
assignment = {}

# Count assignment attempts
attempts = 0


def is_safe(region, colour):
    # Check whether any already-coloured neighbour
    # has the same colour
    for neighbour in neighbours[region]:
        if assignment.get(neighbour) == colour:
            return False

    return True


def solve(index):
    global attempts

    # All regions have been coloured
```

```
    if index == len(regions):
        return True

    # Select the next region
    region = regions[index]

    # Try each available colour
    for colour in colours:
        attempts += 1

        if is_safe(region, colour):
            # Assign the colour
            assignment[region] = colour

            # Solve for the next region
            if solve(index + 1):
                return True

            # Backtrack: remove the colour assignment
            del assignment[region]

    return False


# Run the CSP solver
if solve(0):
    print("Valid Map Colouring:\n")

    for region in regions:
        print(region, ":", assignment[region])

    # Verify all adjacency constraints
    valid = True

    for region in regions:
        for neighbour in neighbours[region]:
            if assignment[region] == assignment[neighbour]:
                valid = False
                print("Conflict:", region, "and", neighbour)
```

```
    print("\nAll adjacency constraints satisfied:", valid)
    print("Total assignment attempts:", attempts)

else:
    print("No valid colouring exists.")
```

Output:

```
Valid Map Colouring:

A : Red
B : Green
C : Blue
D : Red
E : Green
F : Blue

All adjacency constraints satisfied: True

Total assignment attempts: 12
```

Experiment 7 - Implementing the Minimax Algorithm

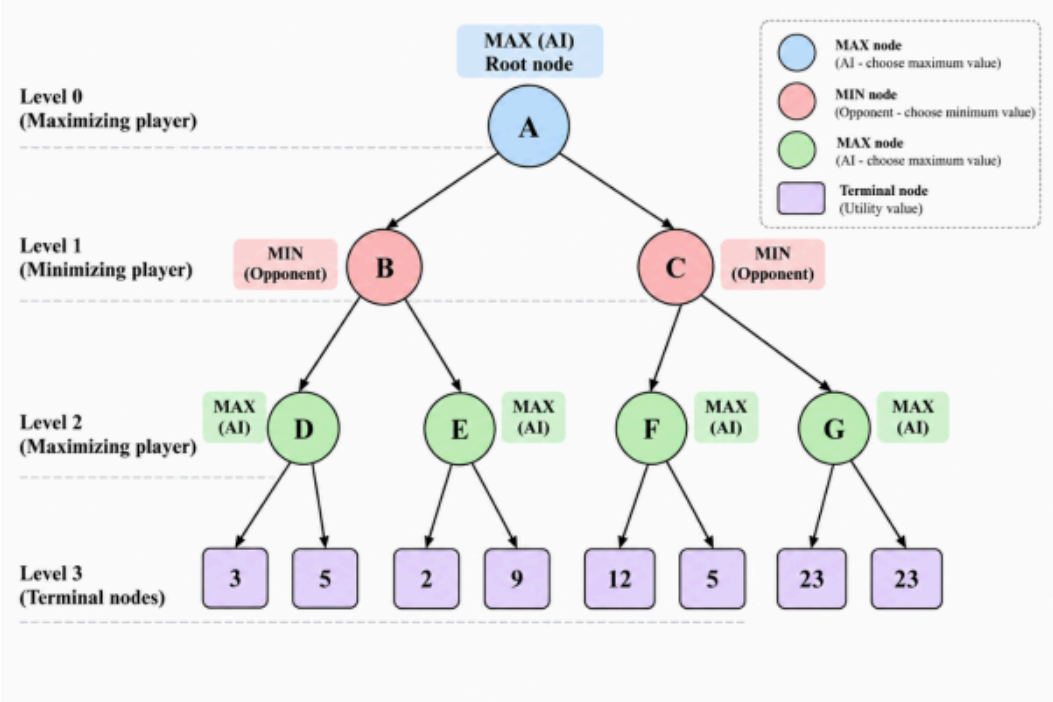
AIM: To demonstrate decision-making in a two-player strategy game.

QUESTION:

A game-playing AI must select its best move in a two-player game where one player tries to maximize the score while the opponent tries to minimize it.

Develop a Python program to implement the Minimax algorithm for a game represented as a binary game tree. The program must:

1. **Represent the game tree with terminal utility values.**
2. **Treat the AI as the maximizing player and the opponent as the minimizing player.**
3. **Recursively evaluate the game tree using the Minimax algorithm.**
4. **Display the evaluated values and the best move selected by the AI.**
5. **Explain how the algorithm assumes that both players choose optimal moves.**



Code:

```
# Experiment 7: Minimax Algorithm

# Game tree based on the given diagram
tree = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F", "G"],
    "D": [3, 5],
    "E": [2, 9],
    "F": [12, 5],
    "G": [23, 23]
}

# Store the evaluated value of each node
values = {}


def minimax(node, is_maximizing):
    # If the node is a terminal node, return its utility
    if isinstance(node, int):
        return node
```

```
    # Evaluate all children recursively
    child_values = []

    for child in tree[node]:
        value = minimax(child, not is_maximizing)
        child_values.append(value)

    # MAX chooses the largest value
    if is_maximizing:
        best_value = max(child_values)
    else:
        # MIN chooses the smallest value
        best_value = min(child_values)

    values[node] = best_value
    return best_value

# Start from root node A (MAX)
best_value = minimax("A", True)

print("Minimax Evaluation:")
for node in ["D", "E", "F", "G", "B", "C", "A"]:
    print(node, "=", values[node])

print("\nBest move for AI:", tree["A"][
    0 if values["B"] >= values["C"] else 1
])
print("Best achievable utility:", best_value)
```

Output:

```
Minimax Evaluation:
D = 5
E = 9
F = 12
G = 23
B = 5
C = 12
A = 12

Best move for AI: C
Best achievable utility: 12
```

Experiment 8 - Implementing Alpha-Beta Pruning

AIM: To optimize decision-making in a two-player strategy game.

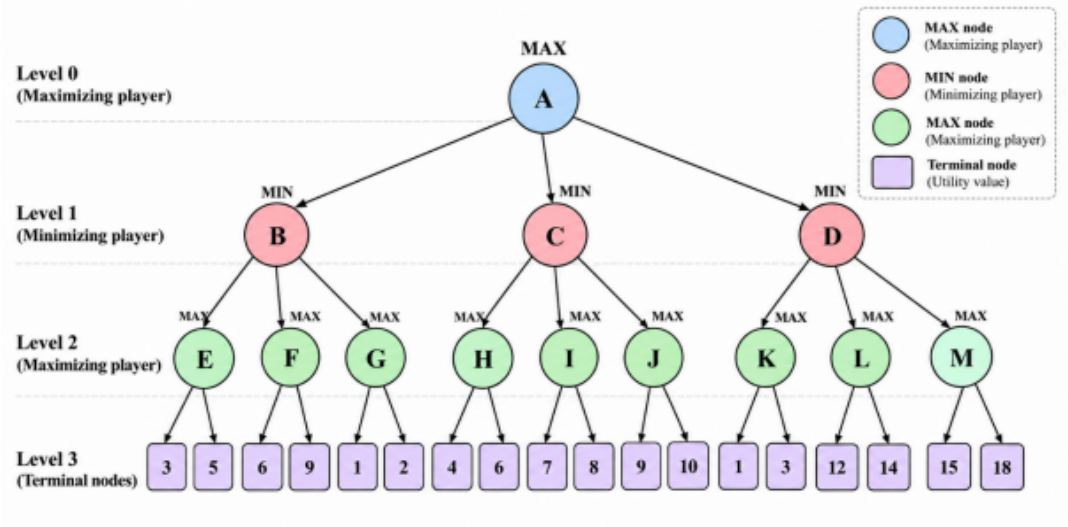
QUESTION:

A chess-playing AI evaluates possible moves by exploring a game tree. Since evaluating every possible move is computationally expensive, the AI uses Alpha-Beta Pruning to eliminate branches that cannot influence the final decision.

Develop a Python program to implement the Alpha-Beta Pruning algorithm for a chess-inspired game tree with multiple possible moves and terminal utility values.

The program must:

1. **Represent the game tree with multiple candidate moves and terminal utility values.**
2. **Implement Alpha-Beta Pruning using alpha and beta bounds.**
3. **Identify and skip branches that cannot affect the final decision.**
4. **Display the selected move, its utility value, the number of nodes evaluated, and the number of branches pruned.**
5. **Explain how Alpha-Beta Pruning reduces the search effort while preserving the optimal decision.**



Code:

```
# Experiment 8: Alpha-Beta Pruning Algorithm
```

```
# Game tree based on the given diagram
tree = {
    "A": ["B", "C", "D"],
    "B": ["E", "F", "G"],
    "C": ["H", "I", "J"],
    "D": ["K", "L", "M"],
    "E": [3, 5],
    "F": [6, 9],
    "G": [1, 2],
    "H": [4, 6],
    "I": [7, 8],
    "J": [9, 10],
    "K": [1, 3],
    "L": [12, 14],
    "M": [15, 18]
}

# Count evaluated terminal nodes and pruned branches
nodes_evaluated = 0
branches_pruned = 0


def alpha_beta(node, alpha, beta, is_max):
    global nodes_evaluated, branches_pruned

    # Terminal node: return its utility value
    if isinstance(node, int):
        nodes_evaluated += 1
        return node

    if is_max:
        best_value = float("-inf")

        for i, child in enumerate(tree[node]):
            value = alpha_beta(child, alpha, beta, False)
            best_value = max(best_value, value)
            alpha = max(alpha, best_value)

            # Prune remaining branches if alpha >= beta
```

```
            if alpha >= beta:
                branches_pruned += len(tree[node]) - i - 1
                break

        return best_value

    else:
        best_value = float("inf")

        for i, child in enumerate(tree[node]):
            value = alpha_beta(child, alpha, beta, True)
            best_value = min(best_value, value)
            beta = min(beta, best_value)

            # Prune remaining branches if beta <= alpha
            if beta <= alpha:
                branches_pruned += len(tree[node]) - i - 1
                break

        return best_value


# Evaluate each candidate move from the root
move_values = {}

for move in tree["A"]:
    move_values[move] = alpha_beta(
        move, float("-inf"), float("inf"), False
    )

# Select the best move for MAX
best_move = max(move_values, key=move_values.get)
best_value = move_values[best_move]

print("Move evaluations:")
for move, value in move_values.items():
    print(move, "=", value)

print("\nBest move for AI:", best_move)
print("Best utility value:", best_value)
```

```
print("Terminal nodes evaluated:", nodes_evaluated)
print("Branches pruned:", branches_pruned)
```

Output:

```
Move evaluations:
B = 2
C = 6
D = 3

Best move for AI: C
Best utility value: 6
Terminal nodes evaluated: 13

Branches pruned: 5
```